

wimsalabim · PyPI README

Captain WimLee115

2026-05-10

wimsalabim

Honest, audit-grade website security and privacy reconnaissance — under the PVNL flag.

license **AGPL-3.0** python 3.10 | 3.11 | 3.12 | 3.13

[Source on GitHub](#) · [Full documentation](#) · [Pentest report](#)

Why

The website-scanner space is full of SaaS tools that send your target list to a third party, big offensive frameworks that assume you know your legal limits, and single-domain CLIs you have to glue together yourself.

`wimsalabim` is a single, opinionated CLI that:

- Runs **fully local** — no SaaS, no telemetry, no analytics, no phone-home.
- Is **passive by default** — active analyzers refuse to run unless authorization is verified.
- Produces **signed, deterministic, machine-readable reports** suitable as forensic evidence.
- Uses a **transparent rule-based risk engine** — every score is explainable to a non-technical reader.
- Ships with **strict typing, tests, and a security audit** so the tool itself meets the standard it asks of its targets.

Install

```
pipx install wimsalabim
```

Or with `pip` in a venv:

```
python -m venv .venv && source .venv/bin/activate  
pip install wimsalabim
```

Quick start

```
# Passive scan on any target
wimsalabim scan example.com

# JSON output for CI
wimsalabim scan example.com --format json -o report.json

# Markdown for PR comments
wimsalabim scan example.com --format markdown

# SARIF 2.1.0 for GitHub Code Scanning
wimsalabim scan example.com --format sarif -o sarif.json

# Active port scan – requires authorization proof
wimsalabim scan example.com \
  --auth-self-owned ./my-domains.txt \
  --enable ports

# Cryptographic integrity
wimsalabim keys init
wimsalabim scan example.com --sign --format json -o signed.json
wimsalabim verify signed.json # → valid (or INVALID on tamper)
```

Built-in analyzers

Analyzer	Class	Purpose
<code>dns_recon</code>	passive	A / AAAA / MX / NS / TXT / SOA / CNAME / CAA + DNSSEC
<code>tls</code>	passive	TLS handshake, leaf-cert validity, expiry, protocol cipher
<code>headers</code>	passive	Security & info-leak headers via single GET
<code>ports</code>	active	Async TCP-connect scan (authorization required)

Output formats

`rich` (terminal · default) · `json` (RFC 8785 canonical) · `markdown` · `sarif` (2.1.0)

Legal compliance — built-in, not a disclaimer

- **NL Sr 138ab** — `AuthorizationGate` refuses active analyzers without verified proof (DNS-TXT, well-known, or self-owned manifest).
- **GDPR / AVG art. 5, 6, 25** — WHOIS-PII redacted by default; no telemetry; bodies hashed (not stored).
- **EU AI-Act** — risk engine is rule-based and explainable, not a learned model.

What you get

- 118 tests, 76% coverage, mypy -strict clean, ruff clean (40+ rule sets).

- Bandit clean (0 high, 0 medium). pip-audit clean (0 known vulnerabilities).
- Reports signed with Ed25519 over RFC-8785 canonical JSON.
- Optional `OpenTimestamps` anchor on the Bitcoin chain for verifiable timestamps.

Documentation

Full README, architecture, legal kader, pentest report, and contributing guide are in the GitHub repo: **<https://github.com/WimLee115/wimsalabim>**

Under the **PVNL** (Privacy Verzet NL) flag — Captain WimLee115.